Cerio

Project Documentation

October 12, 2026

Contents

1 Overview **2**

2 Game rules **2**
- 2.1 Game goals . . . 2
- 2.2 Game mechanics . . . 2
- 2.3 Level generation . . . 2

3 Architecture **3**
- 3.1 Game core . . . 3
- 3.2 Training environment . . . 3

4 Bot **3**
- 4.1 Operation . . . 3
- 4.2 Approaches . . . 3
- 4.3 Comparison of RL and neuroevolution . . . 4

5 Requirements **4**
- 5.1 Performance requirements . . . 4
- 5.2 Evaluation criteria . . . 4
- 5.3 Technology stack . . . 4

6 Schedule **4**

1 Overview

The subject of the project is the development of a two-dimensional platform game that is simply Super Mario (a side-scrolling Mario clone with original graphics), running in a web browser, and, as its main part, the design, training and comparison of two approaches to automatic level completion by a bot: neuroevolution and reinforcement learning (RL).

The game itself runs entirely on the client, in the browser. The neural networks that drive the bot are trained and run on a server, which is written in Python.

2 Game rules

The main goal of the game is to reach the flagpole at the end of the level. The game consists of procedurally generated, side-scrolling levels. Death sends the player back to the start of the level. Each level contains:

- a level entrance and exit (flagpole),
- ground, platforms and blocks the player can stand on and jump between,
- pits and enemies capable of killing the player.

Some levels also contain items, e.g. question blocks with mushrooms. Completing a level activates a checkpoint, so it does not have to be repeated.

2.1 Game goals

Main goal: reaching the flagpole.

Intermediate goals: jumping over pits, avoiding or stomping enemies, collecting power-ups, reaching checkpoints.

Bonus goals: collecting bonus points (coins).

Failure condition: falling into a pit or contact with an enemy while small, which restarts the level.

2.2 Game mechanics

The mechanics follow the original Mario games.

- running left and right, jumping (jump height depends on how long the button is held), crouching, attacking,
- stomping enemies (the set of enemy types is still being decided),
- pits, solid blocks, bricks, question blocks, coins,
- pipes that lead to bonus sub-areas and back,
- power-ups from Super Mario Bros. 1–3: Super Mushroom, Fire Flower, Super Star, Stopwatch, Super Leaf, Tanooki Suit, Frog Suit and Hammer Suit.

Not included: 1-Up, Heart, Bomb, Magic Potion, Magic Whistle, Goomba's Shoe and P-Wing. The game has no P-meter, no Tanooki statue, no level timer and no swimming levels.

Available movement actions: move left, move right, jump, run (sprint), down (crouch, enter pipe), attack (fireball, hammer, tail).

2.3 Level generation

Levels are procedurally generated from segments cut out of the original Mario levels. A segment is a fixed-size block of tiles (16×15 tiles, i.e. 256 × 240 px) with tags (theme, difficulty) and left and right joints (ground height at the edge). Segments are connected only if their joints match. The length of a level is set by the dataset (an average Mario level is about 13 segments, 3360 × 240 px). The same seed gives the same level.

Levels are organized like a machine learning dataset: difficulty tiers (easy, medium, hard), each split into training, validation and test levels with disjoint seeds. The 10 test levels are frozen as files. Each level has metrics (length, gaps, enemies, difficulty score, human time).

Mechanics from the previous draft that do not exist in Mario (dash, wall climbing, stamina, buttons and levers) are removed from the project.

3 Architecture

The game must run in a web browser, so Pygame is not used. The system has two parts.

Client (browser): a TypeScript application drawing the game on an HTML5 Canvas. It handles input, rendering, the fixed-timestep game loop and level loading. The client contains the complete, working game and can be played without the server.

Server (Python): trains the neural networks and stores the models. It exposes the trained bots to the client over a WebSocket connection: the client sends the current observation, the server answers with the next action. The neural network backend may therefore live on the server, while the client only needs a working game.

3.1 Game core

The game logic is a single, deterministic simulation module without any dependency on the rendering layer. The same code runs in the browser (with graphics) and headless on the server side (Node.js) for training. Determinism is ensured by a fixed timestep, integer (sub-pixel) arithmetic and a seeded random number generator. This is a necessary condition for fair comparison of the bots and for reproducible training.

3.2 Training environment

Python reaches the headless core through a Gym-style interface (`reset`, `step` returning observation, reward and done flag). Many environment instances are run in parallel and batched to keep the training throughput high.

Observation: a tile window around the character (solid blocks, enemies, items) and the character state (velocity, size, power-up). No image is used.

Actions: a small discrete set of button combinations (e.g. none, right, right+jump, right+run, right+run+jump, left, jump).

Action repeat: a decision is made every few frames, which also keeps the client–server latency during live play acceptable.

4 Bot

4.1 Operation

The bot reads game metadata instead of an image: the level layout, enemies, items and the character state. Based on this data it selects the next button combination and moves towards the flagpole. The bot completes previously unseen procedural levels.

4.2 Approaches

Both approaches use the same observation, action set, level generator and simulation budget, so their results can be compared directly.

1. **Neuroevolution.** A population of neural network policies is evolved with a genetic algorithm (or NEAT as an alternative). Fitness rewards the distance covered and level completion and penalizes the time used.

2. **Reinforcement learning.** A policy trained with a policy-gradient method in PyTorch. The reward is based on progress to the right, a small time penalty, a death penalty and a bonus for reaching the flagpole.

4.3 Comparison of RL and neuroevolution

The two approaches are compared on:

- the share of completed levels on the test set,
- the completion time relative to the human time,
- training cost: simulation steps and wall-clock time needed to reach the evaluation threshold,
- generalization to unseen levels and variance between training runs (several seeds).

5 Requirements

5.1 Performance requirements

Client: a current desktop browser (Chrome, Firefox or Edge); resolution 854×480 px, maintained at 60 FPS for 80% of the gameplay, assuming the game is "playable".

Server and training: AMD Ryzen 5 7600X (6 cores), NVIDIA GeForce RTX 3070, 48 GB RAM.

5.2 Evaluation criteria

- The bot must complete at least 70% of a test set consisting of 10 levels.
- The bot must complete the task in finite time, shorter than three times the human completion time.
- The criteria are checked for both the neuroevolution and the RL bot, and the results are compared.

5.3 Technology stack

- Client: TypeScript, HTML5 Canvas (no Pygame).
- Headless game core for training: Node.js (the same code as in the client).
- Server and neural networks: Python, PyTorch, WebSocket server (e.g. FastAPI).

6 Schedule

Date	Checkpoint	Scope
08.10.2026	Project requirements	Requirements, game goals, bot assumptions
22.10.2026	Report	Deterministic core, shared config, browser client, headless mode, level format (JSON)
05.11.2026	Milestone I	Blocks, basic enemies, pipes, SMB1 power-ups, environment interface, server link, neuroevolution bot
19.11.2026	Report	SMB3 power-ups, segment import, generator, datasets (train/val/test), RL training pipeline
03.12.2026	Deadline	RL bot, evolution vs RL evaluation, graphics, final report

Table 1: Project checkpoints

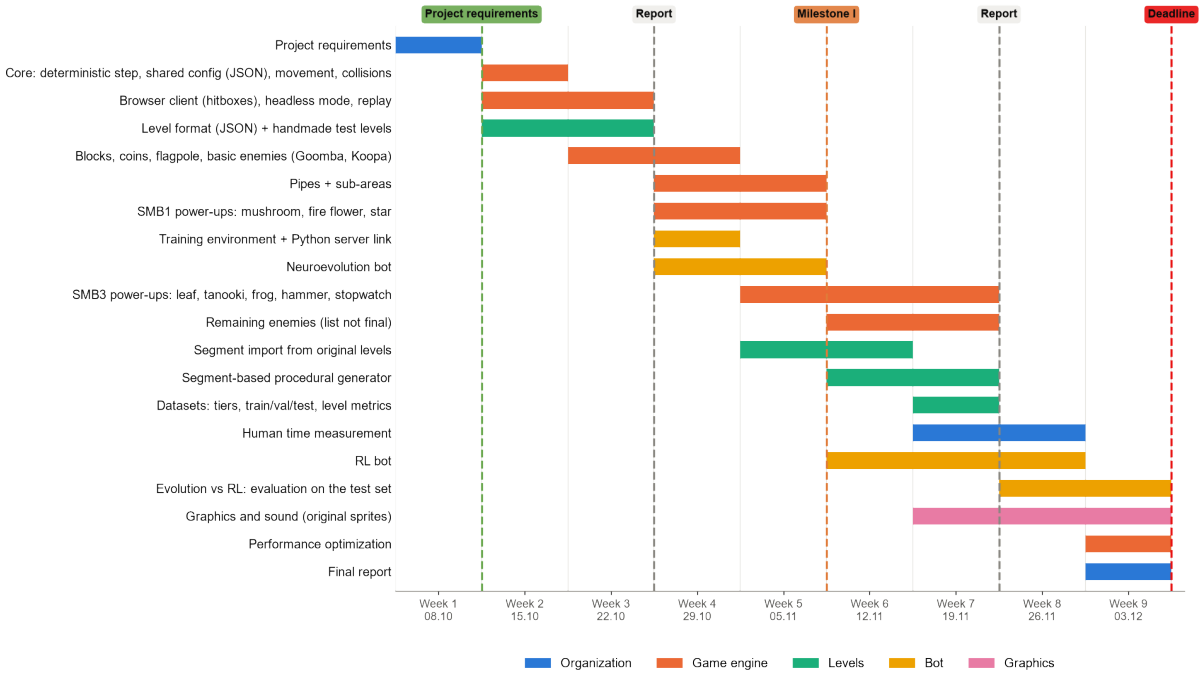


Figure 1: Gantt chart